

CrowPanel Advanced 7inch V1.1

ESP-IDF × Visual Studio Code 導入手順書

対象製品: Elecrow CrowPanel Advanced 7inch ESP32-P4 HMI AI Display 1024×600（Model DHE04107D、**Hardware & Software V1.1**）

対象読者: ESP-IDFを初めてVisual Studio Codeで利用する開発者

作成日: 2026-08-20

作成者: Manus AI

1. 目的と適用範囲

本書は、CrowPanel Advanced 7inch **V1.1** を Windows 10/11 上の Visual Studio Code と ESP-IDF で開発するための、最初の再現可能な導入手順です。最初の目標は、Elecrow公式の **Lesson09-LVGL_Lighting_Control** を改変せずにビルド、書込み、表示、タッチ操作まで確認することです。続いて、同じくV1.1の公式 **Lesson17-Wi-Fi_function / ESP32_P4-station** を単独で動かし、ESP32-P4から搭載ESP32-C6経由で2.4 GHz Wi-Fiへ接続できることを確認します。

重要: 実機はV1.1です。V1.2用のソース、`sdkconfig`、配線情報、画面設定を混在させないでください。本書に記載するパスはすべて `example/V1.1/idf-code/` 配下です。

Elecrowは本製品の開発環境としてESP-IDFとArduino IDEを挙げています。本プロジェクトでは、V1.1用に公式のstationサンプル、P4-C6 Wi-Fi関連コンポーネント、SDK設定が揃っているため、**ESP-IDFを標準開発環境として採用**します。 1 2

本書で行うこと	本書の範囲外
VS CodeとESP-IDF拡張の導入、V1.1公式サンプルのビルド、UART書込み、シリアルログ確認	Raspberry Pi側のManus API / OpenAI API実装
Lesson09でLCD・バックライト・GT911タッチを確認	AI会話UI、WebSocket、JSONイベント処理の実装
Lesson17 stationでWi-Fi接続とIP取得を確認	ESP32-C6のファームウェア更新
Wi-Fi資格情報の安全な扱い、基本的な障害切り分け	V1.2向けのコード移植・設定流用

本書の主手順はWindows 10/11を前提に記載します。macOS/LinuxでもESP-IDF拡張の操作名は基本的に同じですが、シリアルポート名（例: `/dev/cu.*`、`/dev/ttyUSB*`）とパス表記が異なります。 7

2. 作業を始める前の確認

CrowPanel Advanced 7inchはESP32-P4、16 MB Flash、32 MB PSRAM、1024×600 IPSタッチ画面、無線用ESP32-C6を備えます。公式リポジトリは電源として**5 V / 2 A**を示しており、ディスプレイを使うと消費電力が大きくなります。 1 開発PCのUSBポートだけに依存して画面が不安定な場合は、安定した5 V / 2 A以上の電源を用意してください。

準備物	用途・確認事項
CrowPanel Advanced 7inch V1.1	基板または購入記録でV1.1であることを確認します。
USBデータ通信対応ケーブル	充電専用ケーブルではCOMポートが出現しません。
安定した5 V / 2 A以上の電源	画面が消灯・再起動する場合にまず見直します。
Windows 10/11 64-bit PC	本書の画面操作の前提です。
Visual Studio Code	Stable版を使用します。
Git	公式ソースを取得し、後の差分管理に使用します。
2.4 GHz Wi-Fiアクセスポイント	Lesson17のWi-Fi検証時のみ必要です。SSIDとパスワードを準備します。

書込みには、基板上で**UART0のプログラム書込みに対応するUSB-to-UART接続**を使用します。USB2.0機能用ポートや給電専用の接続と取り違えないよう、基板シルク、同梱資料、公式Wikiのポート説明を確認してください。 2 Windowsの「デバイス マネージャー」で接続前後の「ポート（COMとLPT）」の変化を確認する方法が確実です。

2.1 作業フォルダを決める

日本語、空白、クラウド同期フォルダを避けた短いパスを使います。たとえば、以下を作成してください。

Plain Text

```
C:\work\crowpanel-v11\
```

以降、Gitで取得する公式リポジトリは `C:\work\crowpanel-v11\elecrow-official\`、自分で変更する作業コピーは `C:\work\crowpanel-v11\work\` に置くものとします。公式ソースを直接編集しないことで、更新・比較・復旧が容易になります。

3. ESP-IDF開発環境をVisual Studio Codeへ導入する

Espressif公式の推奨手順は、Visual Studio CodeへESP-IDF拡張を追加し、拡張からESP-IDF Installation Managerを起動してフレームワークとツール群を導入する方法です。 7

3.1 Visual Studio CodeとGitを導入する

Visual Studio Codeを[公式サイト](#)から導入します。続けて、[Git for Windows](#)を導入してください。既にGitが利用できる場合は、PowerShellで次のコマンドを実行し、バージョンが表示されることを確認します。

```
Plain Text
```

```
git --version
```

3.2 ESP-IDF拡張を追加する

Visual Studio Codeを起動し、`Ctrl + Shift + X` を押します。拡張機能の検索欄に `ESP-IDF` と入力し、発行元が **Espressif Systems** の **ESP-IDF** 拡張をインストールしてください。 7

次に、`Ctrl + Shift + P` でコマンドパレットを開き、次のコマンドを実行します。

```
Plain Text
```

```
ESP-IDF: Open ESP-IDF Installation Manager
```

インストールマネージャーが起動したら、ESP-IDFと必要ツールを導入します。V1.1公式コードとの最初の互換性確認では、**ESP-IDF v5.4.2系を選択して固定**してください。V1.1公式コース資料が示すESP-IDF 5.4.2以上という条件に合わせ、さらにV1.1のWi-Fiサンプルが `esp_hosted ~2.7.0`、`esp_wifi_remote ^0.16.1` を指定している構成へ揃えるためです。 3 5 6

運用方針: 新しいESP-IDFを使いたい場合も、最初にv5.4.2系で公式Lesson09とLesson17の両方が動くことを確認してください。その後、別フォルダで新しいESP-IDFを試し、動作結果を比較します。最初から最新版へ置き換えると、原因が基板・設定・SDKのどこにあるか切り分けにくくなります。

導入後、コマンドパレットから以下を実行します。

Plain Text

```
ESP-IDF: Select Current ESP-IDF Version
```

一覧から導入した**v5.4.2系**を選択してください。続けて次を実行し、**Error** がないことを確認します。

Plain Text

```
ESP-IDF: Doctor Command
```

ESP-IDF拡張は選択したセットアップの環境変数をワークスペースへ設定します。既存の環境変数を手作業で設定している場合は、選択済みセットアップと競合しないよう、古い **IDF_PATH** などを整理してください。 7

4. Elecrow公式V1.1ソースを取得する

PowerShellを開き、次のコマンドでElecrow公式リポジトリを取得します。

Plain Text

```
New-Item -ItemType Directory -Force C:\work\crowpanel-v11 | Out-Null
Set-Location C:\work\crowpanel-v11
git clone https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-A
```

取得直後に、調査時点のコミットを記録しておく、後日公式リポジトリが更新された場合でも同じソースを再現できます。

Plain Text

```
Set-Location C:\work\crowpanel-v11\elecrow-official
git rev-parse --short HEAD
git status
```

作業対象となるV1.1の公式サンプルは次の2つです。

段階	開くプロジェクト	目的	最初に変更してよいか
1	example\V1.1\idf-code\Lesson09-LVGL_Lighting_Control	LCD、バックライト、GT911タッチ、LVGLの確認	変更しない

2	example\V1.1\idf-code\Lesson17-Wi-Fi_function\ESP32_P4-station	ESP32-C6経由のWi-Fi station接続とIP取得	SSID/パスワードのみ設定
---	--	---------------------------------	----------------

それぞれを `C:\work\crowpanel-v11\work\` 配下へコピーして作業してください。以下はPowerShellの例です。

Plain Text

```
New-Item -ItemType Directory -Force C:\work\crowpanel-v11\work | Out-Null
Copy-Item -Recurse `
  C:\work\crowpanel-v11\elecrow-official\example\V1.1\idf-code\Lesson09-LVGL_Li
  C:\work\crowpanel-v11\work\Lesson09-LVGL_Lighting_Control

Copy-Item -Recurse `
  C:\work\crowpanel-v11\elecrow-official\example\V1.1\idf-code\Lesson17-Wi-Fi_f
  C:\work\crowpanel-v11\work\ESP32_P4-station
```

5. 最初の成功: Lesson09で画面・タッチを確認する

5.1 プロジェクトを開く

Visual Studio Codeで **[File] → [Open Folder...]** を選択し、次のフォルダを開きます。

Plain Text

```
C:\work\crowpanel-v11\work\Lesson09-LVGL_Lighting_Control
```

開くべき場所は、ルートに `CMakeLists.txt` があり、`main` と `peripheral` が並ぶフォルダです。`main` フォルダだけを開くとESP-IDFプロジェクトとして認識されません。既存ESP-IDFプロジェクトはこのようにルートフォルダを開くか、`ESP-IDF: Import ESP-IDF Project` で取り込みます。

8

この公式プロジェクトは、ルートの `CMakeLists.txt` で `EXTRA_COMPONENT_DIRS "peripheral"` を指定し、画面、I2C、タッチ、バックライトのボード固有処理を読み込みます。`peripheral` を削除・移動せず、最初の書込みでは `main/main.c` も変更しないでください。 4

5.2 ターゲットと設定を確認する

V1.1のLesson09に含まれる `sdkconfig` は、ESP32-P4、16 MB Flash、PSRAM、80 MHzフラッシュ、UART0コンソール、115200 bpsを設定済みです。 4 **ESP32-C6をアプリケーションのビル**

ドターゲットに選ばないでください。 本プロジェクトのターゲットはESP32-P4です。

SDK Configuration Editor を開く前に、 sdkconfig 内の次の値を確認できます。

Plain Text

```
CONFIG_IDF_TARGET="esp32p4"  
CONFIG_ESPTOOLPY_FLASHSIZE="16MB"  
CONFIG_ESPTOOLPY_MONITOR_BAUD=115200
```

何らかの理由でターゲットが異なる場合だけ、コマンドパレットからターゲット設定を開き、**ESP32-P4**を選択します。この操作は設定の再生成を伴うことがあるため、その前に作業コピーを取り直せる状態にしておいてください。

5.3 ビルドする

コマンドパレットで、次のコマンドを実行します。

Plain Text

ESP-IDF: Build your Project

ESP-IDF拡張は新しいターミナルにビルドログを表示し、成功時には完了通知を表示します。

9 初回はComponent Managerが、 esp_lcd_ek79007 ^1.0.2 、 esp_lvgl_port ^2.6.0 、 esp_lcd_touch_gt911 ^1.1.3 などの依存コンポーネントを取得します。ネットワーク接続を維持し、完了まで待機してください。 4

ビルドが成功したら、 build フォルダとファームウェアが生成されます。この時点ではまだ基板へ書き込まれていません。

5.4 基板を接続し、COMポートを選ぶ

CrowPanelの**UART0書込み用USB-to-UART接続**を、データ通信対応USBケーブルでPCに接続します。Windowsのデバイス マネージャーで新しいCOMポートを確認します。次にVS Codeのコマンドパレットで実行します。

Plain Text

ESP-IDF: Select Port to Use

表示された一覧から該当する COMx を選択します。ESP-IDF拡張の公式手順でも、書込みとモニタの前にこの操作を行います。 10 COMポートが表示されない場合は、後述の「8. トラブルシューティング」を参照してください。

5.5 書き込む

次のコマンドを実行します。

Plain Text

ESP-IDF: Flash your Project

書き込み方式は**UART**を選択します。ESP-IDF拡張はUART、JTAG、DFUを選択できますが、本手順ではUART書き込みを使用します。**10** 成功すると、ターミナルにフラッシュ処理のログが出て、拡張は完了通知を表示します。

自動でダウンロードモードへ入れない場合は、基板にある **BOOT** と **RESET** の表記を確認してください。一般的なESPの手順は「**BOOT** を押しながら **RESET** を一度押して離し、書き込み開始後に **BOOT** を離す」です。ただし、基板のボタン表記や接続ポートに従うことを優先し、不明な状態で端子を短絡させないでください。

5.6 シリアルモニタと画面を確認する

コマンドパレットから次を実行します。

Plain Text

ESP-IDF: Monitor Device

モニタ速度はプロジェクトの **CONFIG_ESPTOOLPY_MONITOR_BAUD** で定義され、Lesson09は115200 bpsです。**11** シリアルログには、LDO、I2C、タッチ、LCD、バックライト、GPIO48、UIの初期化成功が順に現れます。代表的には次のような語句を確認します。

Plain Text

```
LD03 and LD04 init success
I2C init success
Touch panel init success
LCD init success
LCD backlight opened (brightness: 100)
UI created successfully
System initialized successfully
```

画面には **LED Controller**、**LED ON**、**LED OFF** の2ボタンが表示されます。各ボタンをタッチし、シリアルモニタに **LED turned ON** または **LED turned OFF** が出ることを確認してください。これは画面、タッチ、LVGLイベント、GPIO48の最小一貫動作確認です。**4**

6. 次の成功: Lesson17 stationでWi-Fi接続を確認する

6.1 Wi-Fiの方式を正しく理解する

CrowPanel V1.1では、アプリケーションは**ESP32-P4**で動き、無線機能は搭載された**ESP32-C6**を利用します。V1.1公式stationサンプルは、`esp_hosted ~2.7.0` および `esp_wifi_remote ^0.16.1` を依存関係として明示し、P4-C6間のSDIO設定を `sdkconfig.defaults.esp32p4` へ持ちます。 5 6

結論: この基板の最初のWi-Fi検証では、通常の単一ESP32向け `WiFi.h` を前提にせず、V1.1公式の `ESP32_P4-station` をそのまま使用します。P4-C6の接続に必要な設定を自作・推測しないことが重要です。

`sdkconfig.defaults.esp32p4` には、C6スレーブ有効化、ESP Hosted有効化、SDIO 1-bit、`CMD=GPIO19`、`CLK=GPIO18`、`D0=GPIO14`、`D1=GPIO15`、`C6リセット=GPIO32`が含まれます。このファイルを削除、V1.2版で上書き、他プロジェクトの設定で置換してはいけません。 6

6.2 SSIDとパスワードを設定する

VS Codeで次のプロジェクトを開きます。

Plain Text

C:\work\crowpanel-v11\work\ESP32_P4-station

コマンドパレットから次を実行します。

Plain Text

ESP-IDF: SDK Configuration editor

設定画面で **Example Configuration** を開き、以下を設定します。

設定名	設定内容
WiFi SSID	使用する2.4 GHz Wi-FiのSSID
WiFi Password	上記SSIDのパスワード
Maximum retry	初期検証では既定値の5回で可

このサンプルはSSIDとパスワードをKconfigから設定し、実行時に使用します。 5 デフォルトの `myssid`、`mypassword` のままでは接続できません。

SSIDとパスワードは `sdkconfig` に残るため、次のいずれかを実施してください。

開発形態	推奨対策
------	------

個人PCだけで利用	sdkconfig をローカルに保存し、スクリーンショット・ログ共有時にSSIDを伏せます。
GitHubなどへ公開・共同開発	初回に <code>git rm --cached sdkconfig sdkconfig.old</code> を実行して追跡対象から外し、 <code>.gitignore</code> に <code>/sdkconfig</code> と <code>/sdkconfig.old</code> を追加します。実ファイルは削除せずローカルに残してください。
YouTube撮影	電波名に個人情報を含めないゲストSSIDを使い、シリアルログのSSID・パスワード行を映さない、またはマスクします。

注意: V1.1のstationサンプルは接続成功・失敗時にSSIDとパスワードをログ出力します。これは公式サンプルの疎通確認用挙動です。実アプリケーションへ発展させる際は、このログ出力を削除または伏せ字化してください。 5

6.3 ビルド、書込み、IP取得を確認する

Lesson09と同様に、`ESP-IDF: Build your Project`、`ESP-IDF: Select Port to Use`、`ESP-IDF: Flash your Project`、`ESP-IDF: Monitor Device` の順に実行します。 9 10 11

正しく接続できると、シリアルモニタに次のようなログが出ます。

Plain Text

```
wifi station: ESP_WIFI_MODE_STA
wifi station: wifi_init_sta finished.
wifi station: got ip:192.168.x.x
```

特に **got ip:** が表示されれば、DHCPによるIP取得まで成功です。失敗時は `retry to connect to the AP` の後に `Failed to connect to SSID` が出ます。 5 その場合はSSID、パスワード、2.4 GHz帯、アクセスポイント側のMACフィルタリング、電波強度を順に確認してください。

この段階では画面サンプルとWi-Fiサンプルが**別プロジェクト**であることが正しい状態です。両方が同時に動かないことは異常ではありません。最初に各公式サンプルが単独で成功することを、後の統合作業の基準にします。

7. 合格基準と記録すべき証跡

作業を小さく区切り、成功した構成を残してください。少なくとも、ESP-IDFバージョン、公式リポジトリのコミット、使用COMポート、ログの要点を開発メモへ残すことを推奨します。

段階	合格条件	保存する証跡
環境導入	ESP-IDF: Doctor Command が正常終了する	Doctor出力、ESP-IDF v5.4.2系の表示
Lesson09ビルド	エラーなくビルドが完了する	ビルド終端のログ、 <code>git rev-parse --short HEAD</code>
Lesson09書込み	UART書込みが完了し再起動する	Flash完了ログ、利用COMポート
LCD・タッチ	LED Controller と2ボタンが表示され、タッチログが出る	画面写真、 <code>UI created successfully</code> ログ
Lesson17ビルド	stationプロジェクトがビルドできる	依存取得・ビルドログ
Wi-Fi接続	<code>got ip:</code> が表示される	SSID・パスワードを伏せたモニタログ

8. トラブルシューティング

症状	主な原因	切り分けと対処
COMポートが出ない	充電専用ケーブル、接続先ポート違い、USB-UARTドライバ未認識	データ通信ケーブルへ交換します。デバイス マネージャーで接続前後を比較します。UART0書込み用USB接続を使っているか基板シルクとWikiで再確認します。
書込み中に接続待ちのまま進まない	自動ダウンロードモードに入っていない、別アプリがCOMポートを使用中	シリアルモニタやターミナルを閉じ、ポートを選び直します。それでも失敗する場合に限り、基板表記を確認したうえでBOOT/RESET操作を試します。
画面が暗い、途中で消える、再起動する	電源容量・ケーブル品質不足	PC USB給電依存をやめ、5 V / 2 A以上の安定電源を使用します。画面のためのLDO初期化ログも確認します。 1 4
<code>IDF_TARGET</code> 関連のビルドエラー	ESP32-P4以外を選択、設定を不用意に再生成	<code>CONFIG_IDF_TARGET="esp32p4"</code> を確認し、問題が続く場合は作業コピーを捨てて公式V1.1サンプルを新たにコピーします。

Component Managerの取得失敗	インターネット接続、プロキシ、ESP-IDFバージョンの不一致	ネットワークを確認後、v5.4.2系を選び直します。 idf_component.yml のコンポーネントを手作業で別バージョンへ変更しません。
Lesson09で画面表示はあるがタッチが反応しない	peripheral の欠落・改変、初期化失敗	peripheral フォルダとルート CMakeLists.txt の EXTRA_COMPONENT_DIRS "peripheral" を復元します。 Touch panel init success をログで確認します。 4
Wi-Fiが接続できない	SSID/パスワード誤り、5 GHz専用SSID、弱い電波	Example Configuration を見直します。2.4 GHzを有効にし、ログの retry to connect から開始しているか確認します。 5
Wi-Fiのために WiFi.h を追加したが動かない	P4-C6構成を一般的な単一チップ ESP32と誤認	追加を取り消し、公式 ESP32_P4-station を無変更で成功させます。 esp_hosted 、 esp_wifi_remote 、 V1.1の sdkconfig.defaults.esp32p4 を維持します。 5 6
画面とWi-Fiを1プロジェクトにしたら両方が動かない	画面側とstation側のSDK設定・依存関係の統合不足	統合を中断し、Lesson09とLesson17を個別に再検証します。統合時は次節の順序を守り、V1.2設定を混ぜません。

9. AI会話表示プロジェクトへ進むための次の手順

Lesson09とLesson17を別々に成功させた後、初めて crowpanel_ai_display_wifi の作成へ進みます。ここでは完成コードを作らず、統合時に守るべき順序だけを定義します。

順序	実施内容	守るべき事項
1	Lesson09を新しい作業プロジェクトへ複製する	peripheral とルート CMakeLists.txt を維持します。
2	stationのWi-Fi設定を統合する	sdkconfig.defaults.esp32p4 、 sdkconfig.defaults 、 esp_hosted 、 esp_wifi_remote を

		V1.1 stationから慎重に取り込みます。 5 6
3	LCD・タッチ・Wi-Fiの同時動作を確認する	まだWebSocketやAI会話UIを追加しません。
4	Raspberry Piの最小HTTPサーバーへ接続する	hello や ping など短い固定メッセージだけを使います。
5	WebSocket、JSON、LVGL会話カードを追加する	WebSocket受信から直接LVGLを操作せず、キュー経由でUI側が更新する設計にします。
6	PiのManus × GPTオーケストレーターと接続する	Manus/OpenAIのAPIキー、会話履歴、ログ、リトライはRaspberry Piに限定します。CrowPanelへクラウドAPIキーを置きません。

Wi-Fi統合時に**ESP32-C6のファームウェアを書き換える必要はありません**。本書の対象はP4アプリケーションへV1.1公式のリモートWi-Fiスタックを使わせることです。C6ファームウェア更新は、公式資料で対象ハードウェア版、更新手順、復旧手段を確認できた場合にのみ、別作業として実施してください。

10. 公式確認済み事項と実機で確認する事項

区分	内容
公式資料・公式ソースで確認済み	製品はESP-IDFに対応し、V1.1にLesson09とLesson17 stationのプロジェクトが存在し、公式コース資料はESP-IDF 5.4.2以上を導入条件として示します。 1 3
公式ソースで確認済み	Lesson09はEK79007 LCD、GT911タッチ、esp_lvgl_port を使います。Lesson17は esp_hosted 、 esp_wifi_remote とV1.1のP4-C6 SDIO設定を使います。 4 5 6
本書の推奨	最初はESP-IDF v5.4.2系を固定し、公式サンプルを無変更で個別検証してから統合します。

実機で確認が必要

PCで認識されるCOMポート、USBケーブル品質、電源の安定性、アクセスポイントとの相性、基板上のBOOT/RESET操作の必要性です。

参考資料

- [1] Elecrow公式GitHub: CrowPanel Advanced 7inch ESP32-P4 HMI AI Display
- [2] Elecrow公式Wiki: CrowPanel Advanced 7inch
- [3] Elecrow公式コース資料: Advance HMI P4 7inch Course V1.1
- [4] Elecrow V1.1 Lesson09-LVGL_Lighting_Control
- [5] Elecrow V1.1 Lesson17 ESP32_P4-station
- [6] Elecrow V1.1 station: sdkconfig.defaults.esp32p4
- [7] Espressif: Install ESP-IDF and Tools
- [8] Espressif: Create or open an ESP-IDF Project
- [9] Espressif: Build Your Project
- [10] Espressif: Flash onto the Device
- [11] Espressif: Monitor the Output

改訂履歴

版	日付	内容
1.0	2026-08-20	V1.1公式ソース、Elecrow公式資料、Espressif公式VS Code拡張ドキュメントを照合して初版作成。